

PROJECT REPORT

HOTEL MANAGEMENT SYSTEM (HMS)

Course: Advanced Database Management System (ADBMS)
Developer: Aryan Patel
Technology Stack: React.js, Node.js, Express.js, MongoDB, Mongoose ODM
Database Tooling: MongoDB Compass, Postman

1. Introduction & Project Overview

1.1 Objective

The primary objective of this project is to develop a full-stack Hotel Management System designed to handle daily hotel operations—room allocation, guest profile tracking, reservation lifecycle, front-desk check-in/check-out, billing/invoicing, and staff administration—while implementing core Advanced Database Management System (ADBMS) principles.

1.2 Academic Scope (ADBMS Focus)

Unlike traditional relational database projects that rely purely on SQL foreign keys and tabular normalization, this system utilizes MongoDB (a document-oriented NoSQL database) and Mongoose ODM to demonstrate:

- 1. Document-Based Data Modeling:** Managing structured yet flexible schemas with nested subdocuments, data validation constraints, and enums.
- 2. Normalized Referencing vs. Embedding:** Strategically referencing documents across collections using `ObjectId` references and joining them dynamically using `$lookup` and Mongoose `.populate()`.
- 3. Compound and Single-Field Indexing:** Creating single and compound B-tree indexes to optimize query execution and enforce business logic (such as preventing duplicate bookings and unique contact identifiers).
- 4. Multi-Stage Aggregation Pipelines:** Executing complex, in-database analytics using aggregation stages (`$facet`, `$group`, `$lookup`, `$unwind`, `$project`, `$dateToString`, `$sort`) rather than fetching large datasets into the application server's memory.

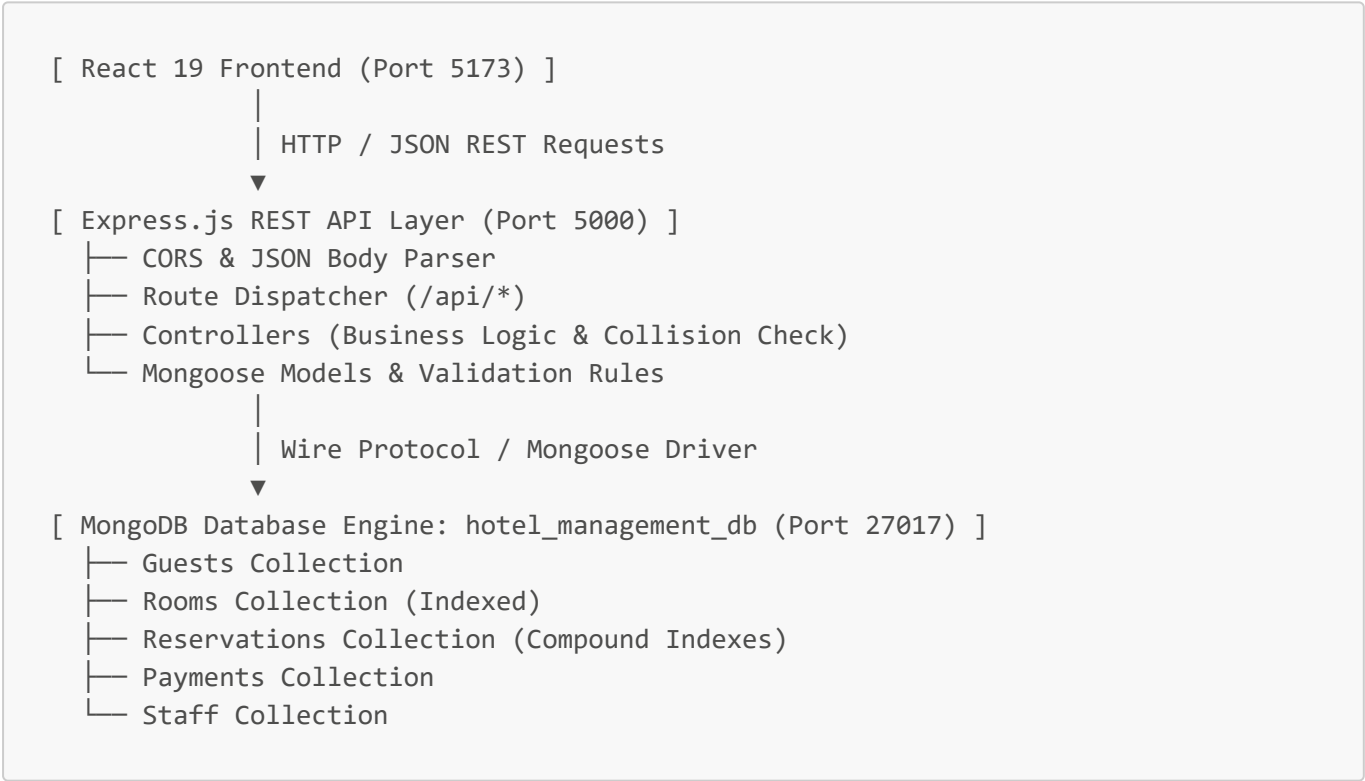
2. Technology Stack & Environment Details

Layer	Technology	Version / Tool	Role in Architecture
Frontend	React.js	React 19 + Vite	Single Page Application (SPA), state management, responsive user interface
Styling	Tailwind CSS	v4.0	Utility-first CSS styling, custom color-coding for room statuses
Icons	Lucide React	v1.52	Visual iconography across modules

Layer	Technology	Version / Tool	Role in Architecture
Backend	Node.js / Express	Node v24, Express v4.21	RESTful API server, routing, request validation, business logic
Database	MongoDB Server	Community Edition (Port 27017)	Document store for hotel operational records
ODM	Mongoose	v8.12	Schema definitions, model validation, query middleware, aggregations
API Testing	Postman	JSON Collection v2.1	Endpoint validation, payload testing, status code checks
GUI	MongoDB Compass	Local GUI Client	Direct visualization of collections, indexes, and document structures

3. System Architecture & Request Lifecycle

The application follows a modular **3-Tier Client-Server Architecture**:



Request-Response Data Flow

- User Action:** A front-desk staff member books a room or performs a check-in on the React UI.
- Client Dispatch:** The component calls a service function in `frontend/src/services/api.js` using Axios.
- Vite Proxy:** The request is sent to `/api/...`, which the Vite dev server proxies to `http://localhost:5000`.
- Server Routing:** `backend/src/server.js` receives the request and routes it to the matching router module in `backend/src/routes/`.

5. **Controller Processing:** The controller (`backend/src/controllers/`) parses input, runs validation, executes Mongoose queries or aggregation pipelines, and handles errors.
6. **Database Execution:** MongoDB executes the query against indexed collections or runs the aggregation pipeline in its internal engine.
7. **Response Delivery:** A structured JSON object (`{ success: true, data: ... }`) is returned to the client, updating UI state in real-time.

4. Codebase Directory Structure: "What Is Happening & Where"

The codebase is organized into separated `backend` and `frontend` folders:

```

Hotel-management-ADBMS/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   └── db.js                # MongoDB connection setup and connection
│   ├── error listeners
│   │   ├── models/
│   │   └── Guest.js            # Guest schema with regex email validation &
│   │                                   ID proof subdocument
│   │   ├── Room.js            # Room inventory schema with unique
│   │   │   roomNumber & status enums
│   │   └── Reservation.js      # Reservation schema with ObjectId refs and
│   │                                   compound index
│   │   ├── Payment.js         # Invoice schema with tax breakdown, payment
│   │   │   method, and date index
│   │   └── Staff.js           # Staff roster schema with shift schedules
│   │                                   and employee code
│   │   ├── controllers/
│   │   │   ├── guestController.js # Guest CRUD, search filter, and populated
│   │   │   │   booking history
│   │   │   ├── roomController.js  # Room filtering, availability checking, and
│   │   │   │   quick status patch
│   │   │   ├── reservationController.js # Booking creation with date collision
│   │   │   │   check & cancellation
│   │   │   ├── checkInOutController.js # Arrival check-in, departure checkout &
│   │   │   │   instant bill generation
│   │   │   ├── paymentController.js # Payment invoice creation, query filters,
│   │   │   │   and status updater
│   │   │   └── staffController.js  # Staff roster CRUD and auto-generated staff
│   │   │   │   code
│   │   │   └── dashboardController.js # Multi-stage MongoDB aggregation pipelines
│   │   │   │   for KPIs and analytics
│   │   └── routes/
│   │       ├── guestRoutes.js    # Endpoints mounted at /api/guests
│   │       ├── roomRoutes.js     # Endpoints mounted at /api/rooms
│   │       ├── reservationRoutes.js # Endpoints mounted at /api/reservations
│   │       ├── checkInOutRoutes.js # Endpoints mounted at /api/checkinout
│   │       ├── paymentRoutes.js  # Endpoints mounted at /api/payments
│   │       ├── staffRoutes.js    # Endpoints mounted at /api/staff
│   │       └── dashboardRoutes.js # Endpoints mounted at /api/dashboard

```

```

| | | |─ middleware/
| | | |   └─ errorMiddleware.js      # 404 Route Not Found and centralized Error
Handler
| | | |─ seed/
| | | |   └─ seedData.js             # Comprehensive sample data generation
script
| | |   └─ server.js                 # Express server configuration, middleware
stack, port listener
| |   └─ .env                       # Backend environment variables (PORT,
MONGODB_URI)
| |   └─ package.json                # Backend dependencies and runner scripts
| |   └─ hotel_management_postman_collection.json # Ready-to-import Postman test
collection
|
|─ frontend/
| |─ src/
| | |─ components/
| | | |─ Navbar.jsx                 # Top navigation bar, branding, live MongoDB
status badge
| | | |─ Sidebar.jsx                # Module navigation sidebar with module tag
badges
| | | |─ DashboardView.jsx          # Module 8: KPI cards, room status
distribution, recent activity
| | | |─ RoomsView.jsx              # Module 2: Room cards, status toggle,
filters, add room modal
| | | |─ GuestsView.jsx             # Module 1: Guest table, regex search,
booking history modal
| | | |─ ReservationsView.jsx       # Module 3: Booking wizard, collision check,
cancel booking modal
| | | |─ CheckInOutView.jsx         # Modules 4 & 5: Front desk arrivals, in-
house list, checkout bill modal
| | | |─ PaymentsView.jsx           # Module 6: Invoices table, status filters,
printable invoice modal
| | | |─ StaffView.jsx              # Module 7: Staff directory, shift
schedules, add staff modal
| | | |─ AdbmsLabView.jsx           # ADBMS Query Lab: Live pipeline runner with
JSON inspection
| | | |─ services/
| | | |   └─ api.js                 # Centralized Axios client for all backend
REST endpoints
| | |   └─ App.jsx                  # Main layout container and active module
view router
| | |   └─ index.css                # Tailwind CSS v4 styling rules
| | |   └─ main.jsx                 # React DOM entry point
| |   └─ vite.config.js             # Vite build configuration and proxy setup
to port 5000
| |   └─ package.json               # Frontend dependencies (React 19, Tailwind
v4, Lucide)
|
|─ README.md                        # Project documentation and execution
instructions

```

5. Detailed Breakdown of the 8 Functional Modules

Module 1: Guest Management

- **Where:** `backend/src/controllers/guestController.js` and `frontend/src/components/GuestsView.jsx`
- **What Happens:**
 - Manages hotel guest profiles (`firstName`, `lastName`, `email`, `phone`, `address`, `idProof`, `specialRequests`).
 - **Data Validation:** Enforces strict regex validation on email format (`/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/`) and unique email indexing.
 - **Search:** The GET endpoint supports case-insensitive regex search (`$regex`, `$options: 'i'`) across name, email, phone, and ID proof number in a single query.
 - **History Population:** Fetching a guest by ID uses Mongoose `.populate('room')` on the `Reservation` model to return their complete past and active stays.
 - **Deletion Safety:** Before deleting a guest, the controller checks whether any active reservation (`Confirmed` or `CheckedIn`) exists, preventing orphaned bookings.

Module 2: Room Management

- **Where:** `backend/src/controllers/roomController.js` and `frontend/src/components/RoomsView.jsx`
- **What Happens:**
 - Manages hotel room inventory. Each room has a unique `roomNumber`, `roomType` (Single, Double, Deluxe, Suite, Penthouse), `pricePerNight`, `floor`, `capacity`, `amenities`, and `status`.
 - **Room Statuses:** Four controlled states: `Available`, `Occupied`, `Cleaning`, `Maintenance`.
 - **Direct Status Toggle:** Front desk and housekeeping can patch a room's status with one click (e.g., from `Cleaning` to `Available`).
 - **Availability Range Algorithm:** The endpoint `/api/rooms/available?checkInDate=...&checkOutDate=...` queries for rooms that do **not** have overlapping reservations in that time window.

Module 3: Reservation Management & Collision Prevention

- **Where:** `backend/src/controllers/reservationController.js` and `frontend/src/components/ReservationsView.jsx`
- **What Happens:**
 - Creates and manages bookings linking a `Guest` document to a `Room` document.
 - Generates a unique, human-readable booking reference (e.g., `RES-2026-101`).
 - **Collision Prevention Logic:** Before inserting a booking, MongoDB executes an overlap query:

```
const conflictingBooking = await Reservation.findOne({
  room: roomId,
  status: { $in: ['Confirmed', 'CheckedIn'] },
  $or: [
    { checkInDate: { $lt: requestedCheckOut }, checkOutDate: { $gt:
      requestedCheckIn } }
```

```
]
});
```

If any overlapping active reservation exists, the API rejects the request with HTTP 400 and an informative error message.

- **Stay Calculation:** The controller computes the exact number of nights from the dates and calculates `totalAmount = totalNights * room.pricePerNight`.
- **Cancellation Workflow:** When a booking is cancelled, its status updates to `Cancelled`, the reason is logged, and the assigned room is automatically reverted to `Available` if it was locked.

Module 4: Check-In Management

- **Where:** `backend/src/controllers/checkInOutController.js` and `frontend/src/components/CheckInOutView.jsx`
- **What Happens:**
 - Front-desk staff view today's expected arrivals under the Arrivals tab.
 - Clicking **"Check-In Guest Now"** executes an atomic update:
 1. Sets `reservation.status = 'CheckedIn'`.
 2. Records the timestamp in `reservation.actualCheckInTime`.
 3. Updates the associated room's status from `Available` to `Occupied`.
 - Both collections are updated synchronously, ensuring Compass and the dashboard reflect current occupancy.

Module 5: Check-Out Management & Room Release

- **Where:** `backend/src/controllers/checkInOutController.js` and `frontend/src/components/CheckInOutView.jsx`
- **What Happens:**
 - Displays all currently in-house guests (`CheckedIn` status).
 - Clicking **"Check-Out & Settle Bill"** opens the checkout modal with pre-calculated base room charges.
 - Allows staff to append incidental charges (food, beverages, laundry) or subtract discounts.
 - Computes 12% GST on room charges.
 - On confirmation:
 1. Sets `reservation.status = 'CheckedOut'`.
 2. Records `reservation.actualCheckOutTime`.
 3. Creates a final `Payment` record with status `Paid`.
 4. **Releases the Room:** Changes `room.status = 'Cleaning'` so housekeeping is notified to clean the room before it returns to `Available`.

Module 6: Payment & Billing Management

- **Where:** `backend/src/controllers/paymentController.js` and `frontend/src/components/PaymentsView.jsx`
- **What Happens:**
 - Tracks all financial transactions with itemized breakdowns (`roomCharges`, `taxAmount`, `serviceCharges`, `discountAmount`, `totalAmount`).

- Supports payment methods: **Credit Card**, **Debit Card**, **UPI**, **NetBanking**, and **Cash**.
- Generates a unique invoice number (e.g., **INV-2026-801**).
- **Printable Modal Invoice:** In the UI, clicking "**View Invoice**" displays a formatted tax invoice with hotel header, GSTIN, guest information, itemized stay charges, payment stamp, and a one-click print function (**window.print()**).

Module 7: Staff Management

- **Where:** **backend/src/controllers/staffController.js** and **frontend/src/components/StaffView.jsx**
- **What Happens:**
 - Manages hotel employees across six departments: **Manager**, **Receptionist**, **Housekeeping**, **Chef**, **Security**, **Maintenance**.
 - Auto-increments employee identification codes (**EMP-001**, **EMP-002**, etc.).
 - Tracks shift schedules (**Morning (6 AM - 2 PM)**, **Evening (2 PM - 10 PM)**, **Night (10 PM - 6 AM)**, **General (9 AM - 6 PM)**) and monthly salary records.

Module 8: Live Aggregation Dashboard

- **Where:** **backend/src/controllers/dashboardController.js** and **frontend/src/components/DashboardView.jsx**
- **What Happens:**
 - Computes high-level KPIs dynamically using MongoDB aggregation pipelines:
 - **Occupancy Rate:** Calculated directly via **\$facet** count of **Occupied** vs. **totalRooms**.
 - **Total Settled Revenue:** Aggregated using **\$match: { paymentStatus: 'Paid' }** and **\$group: { \$sum: '\$totalAmount' }**.
 - **Inventory Breakdown:** Real-time counts of **Available**, **Occupied**, **Cleaning**, and **Maintenance** rooms.
 - **Room Category Pricing:** Average price per night grouped by room category.
 - **Today's Operational Counts:** Expected arrivals and departures computed against the current calendar day.

6. Advanced Database Management System (ADBMS) Technical Analysis

6.1 Database Schema Modeling & Normalization Decision

The database utilizes five collections: **guests**, **rooms**, **reservations**, **payments**, and **staffs**.

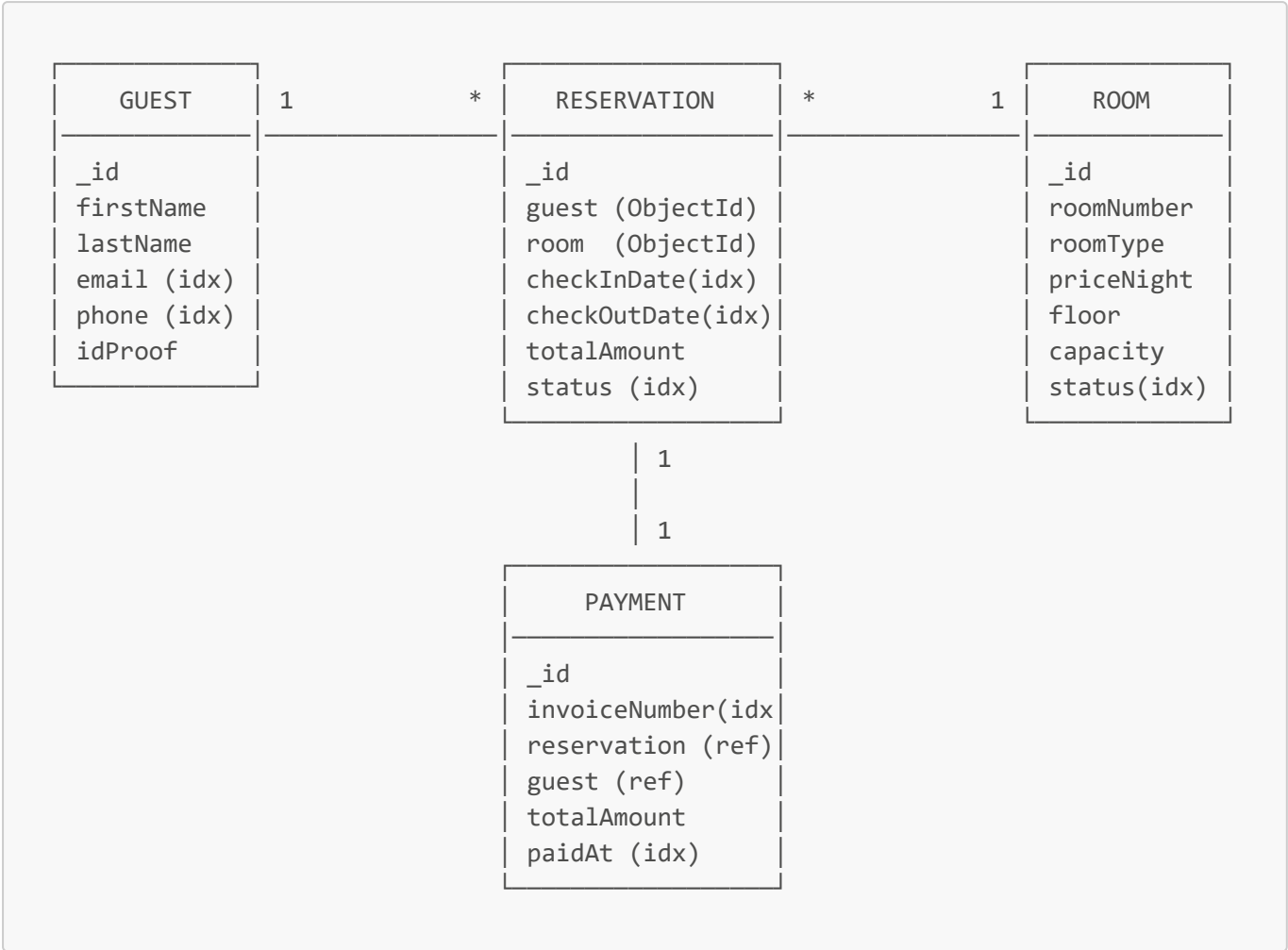
Why Normalization with References was Chosen:

In a hotel system, embedding reservations inside guest documents causes two major NoSQL anti-patterns:

1. **Unbounded Document Growth:** Frequent visitors would cause the guest document to exceed MongoDB's 16MB document size limit over time.
2. **Room Contention Queries:** Checking room availability would require traversing all guest documents.

By storing **reservations** as a separate collection containing **ObjectId** references to **guest** and **room**, the system achieves:

- Independent indexing of dates and room IDs.
- Direct querying for room availability.
- Easy population of related data via Mongoose `.populate()` or MongoDB `$lookup`.



6.2 Indexing Strategy & Performance Justification

The following indexes were created to optimize query performance and enforce uniqueness:

Collection	Indexed Fields	Index Type	ADBMS Justification
rooms	{ roomNumber: 1 }	Unique Single Field	Guarantees no two rooms have identical numbers; supports $O(1)$ lookups.
rooms	{ status: 1, roomType: 1 }	Compound Index	Optimizes front-desk filter queries when selecting available rooms by category.
guests	{ email: 1 }	Unique Single Field	Prevents duplicate profile registrations; enables fast login/lookup.
guests	{ phone: 1 }	Single Field	Speeds up front-desk lookups by guest contact number.
reservations	{ room: 1, checkInDate: 1, checkOutDate: 1 }	Compound Index	Critical for collision avoidance: Allows the database to evaluate date overlaps using an

Collection	Indexed Fields	Index Type	ADBMS Justification
			index scan (IXSCAN) rather than scanning all reservations (COLLSCAN).
reservations	{ guest: 1, status: 1 }	Compound Index	Optimizes retrieval of a guest's past and active booking history.
payments	{ invoiceNumber: 1 }	Unique Single Field	Enforces unique invoice reference numbers.
payments	{ paidAt: -1, paymentStatus: 1 }	Compound Index	Speeds up date-bounded financial aggregations and monthly revenue calculations.
staffs	{ staffCode: 1 }, { email: 1 }	Unique Single Field	Enforces unique employee codes and email addresses.

6.3 In-Depth Analysis of MongoDB Aggregation Pipelines

The application implements three major aggregation pipelines demonstrated in the **ADBMS Query Lab** (</api/dashboard/adbms-lab>):

Pipeline 1: Multi-Faceted Room Inventory (**\$facet**, **\$group**)

Target Collection: **rooms**

Purpose: Executes multiple independent aggregations on room inventory in a single round-trip to the database engine.

```
[
  {
    $facet: {
      statusCounts: [
        { $group: { _id: '$status', count: { $sum: 1 } } }
      ],
      typeDistribution: [
        { $group: { _id: '$roomType', count: { $sum: 1 } } }
      ],
      averagePricing: [
        { $group: { _id: '$roomType', avgPrice: { $avg: '$pricePerNight' } } }
      ]
    }
  }
]
```

- **Execution Explanation:** The **\$facet** stage processes three sub-pipelines concurrently:
 1. **statusCounts:** Counts how many rooms are currently Available, Occupied, Cleaning, or under Maintenance.
 2. **typeDistribution:** Counts room inventory partitioned by category (Single, Double, Deluxe, Suite, Penthouse).

3. **averagePricing**: Uses **\$avg** to calculate the mean price per night per room tier.

Pipeline 2: Top Revenue Generating Clients (**\$lookup**, **\$unwind**, **\$group**, **\$sort**, **\$limit**)

Target Collection: **payments**

Purpose: Joins financial records with guest identity to identify top spenders.

```
[
  { $match: { paymentStatus: 'Paid' } },
  {
    $group: {
      _id: '$guest',
      totalSpent: { $sum: '$totalAmount' },
      transactions: { $sum: 1 }
    }
  },
  { $sort: { totalSpent: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: 'guests',
      localField: '_id',
      foreignField: '_id',
      as: 'guestInfo'
    }
  },
  { $unwind: '$guestInfo' },
  {
    $project: {
      guestName: { $concat: ['$guestInfo.firstName', ' ', '$guestInfo.lastName'] },
      email: '$guestInfo.email',
      totalSpent: 1,
      transactions: 1
    }
  }
]
```

- **Execution Explanation:**

1. **\$match**: Filters only settled payments (**paymentStatus**: 'Paid').
 2. **\$group**: Groups payments by guest **ObjectId**, calculating total money spent (**\$sum**) and transaction count.
 3. **\$sort** & **\$limit**: Sorts descending and keeps the top 5 spenders before performing joins.
 4. **\$lookup**: Performs a left outer join with the **guests** collection to fetch client names.
 5. **\$unwind**: Deconstructs the single-element **guestInfo** array into an object.
 6. **\$project**: Reshapes the document, concatenating first and last names into **guestName**.
-

Pipeline 3: Room Category Performance Analytics (**\$lookup**, **\$unwind**, **\$group**, **\$sort**)

Target Collection: `reservations`

Purpose: Correlates bookings with room types to determine occupancy demand and revenue by room tier.

```
[
  {
    $lookup: {
      from: 'rooms',
      localField: 'room',
      foreignField: '_id',
      as: 'roomDetails'
    }
  },
  { $unwind: '$roomDetails' },
  {
    $group: {
      _id: '$roomDetails.roomType',
      totalBookings: { $sum: 1 },
      totalNightsBooked: { $sum: '$totalNights' },
      totalRevenueGenerated: { $sum: '$totalAmount' },
      averageStayNights: { $avg: '$totalNights' }
    }
  },
  { $sort: { totalRevenueGenerated: -1 } }
]
```

- **Execution Explanation:**

1. `$lookup` links each reservation to its corresponding room document.
2. `$group` groups by `roomType`, calculating total reservations, cumulative nights booked, gross revenue, and mean stay length.
3. `$sort` orders the categories from highest to lowest revenue yield.

7. Testing, Verification & Sample Data

7.1 Seeded Test Data Summary

Running `node src/seed/seedData.js` populates realistic, relational records across all collections:

- **10 Rooms:** Spread across Floors 1 to 4 covering Single (₹1,800), Double (₹2,800 - ₹3,000), Deluxe (₹4,500), Suite (₹7,500), and Penthouse (₹15,000).
- **5 Registered Guests:** Complete with simulated Aadhar cards, Passports, and Driving Licenses.
- **6 Staff Members:** Assigned across Manager, Receptionist, Housekeeping, Chef, and Security roles with morning, evening, and night shifts.
- **Realistic Bookings:** Active in-house stays, upcoming confirmed reservations, completed past stays, and associated GST payment invoices totaling ₹84,700 in initial revenue.

7.2 Postman Test Suite

The included `backend/hotel_management_postman_collection.json` contains predefined test requests for:

- Health check verification (`GET /api/health`).
 - Guest creation and query testing (`POST /api/guests`, `GET /api/guests`).
 - Room availability range query (`GET /api/rooms/available`).
 - Check-in execution (`POST /api/checkinout/check-in/:id`).
 - Check-out with billing breakdown (`POST /api/checkinout/check-out/:id`).
 - Live Aggregation Pipeline inspection (`GET /api/dashboard/adbms-lab`).
-

8. Conclusion

The developed Hotel Management System successfully meets the requirements of the Advanced Database Management System (ADBMS) curriculum:

1. **Full Operational Flow:** Front-desk personnel can manage guests, assign rooms, check guests in, process departures with automated invoice calculation, and oversee staff.
2. **Database Integrity:** Double-booking conflicts are prevented via compound query index constraints, and document references ensure consistent relational integrity.
3. **Database Performance:** Dashboard metrics and analytical reports leverage MongoDB's native aggregation pipeline engine rather than expensive client-side transformations.
4. **Academic Presentation Ready:** The system includes a live **ADBMS Query Explorer** where professors and evaluators can view pipeline definitions and their real-time JSON execution results directly within the application interface.